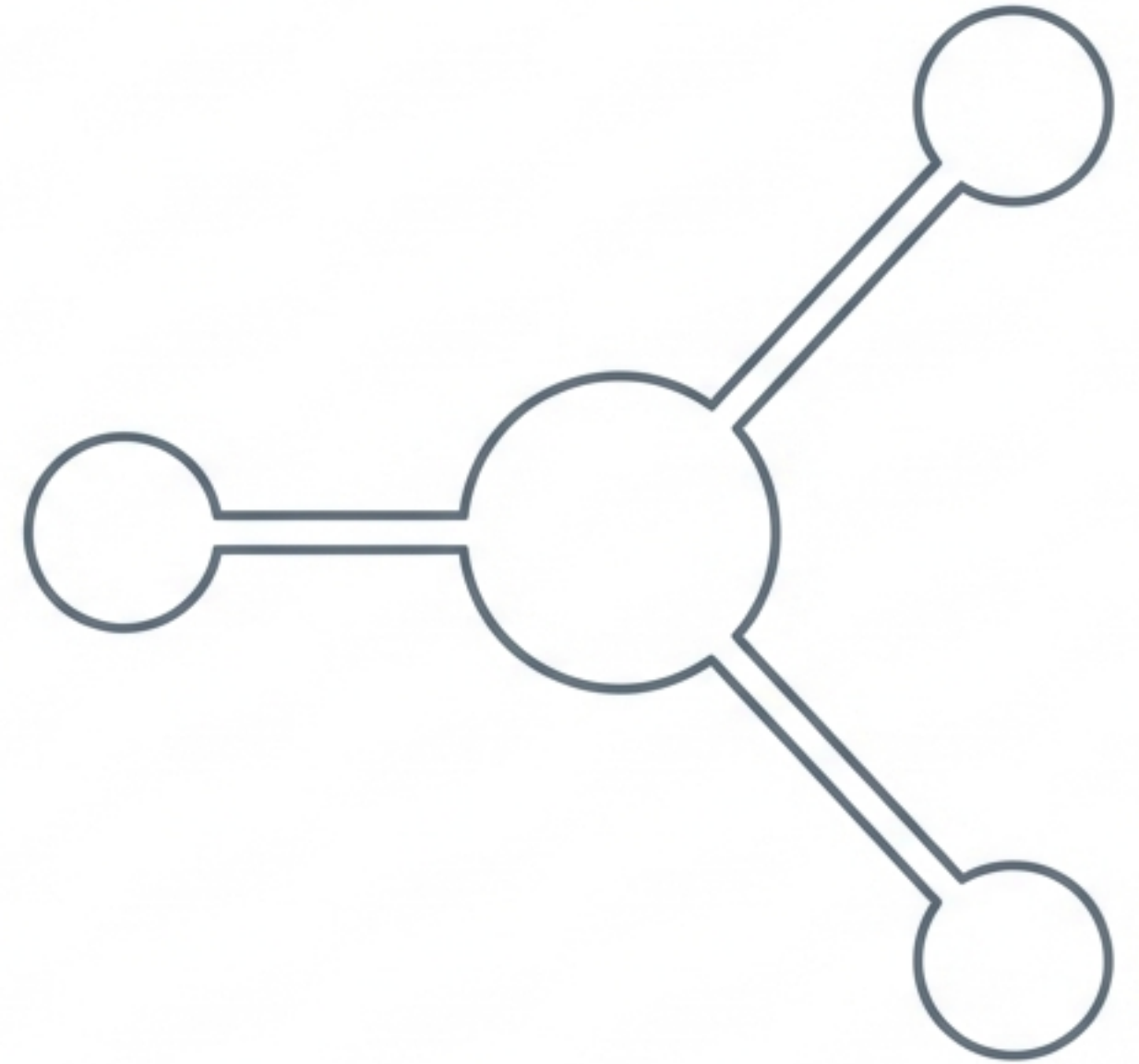


microGPT Architecture — End-to-End Explained

The Irreducible Core of a Transformer



Inspired by Andrej Karpathy's dependency-free implementation.

Why microGPT?

- Pure Python (no NumPy, no PyTorch)
- ~243 lines of code
- 4,192 parameters
- Pedagogical artifact designed for understanding
- Shows the “algorithmic core” of GPT

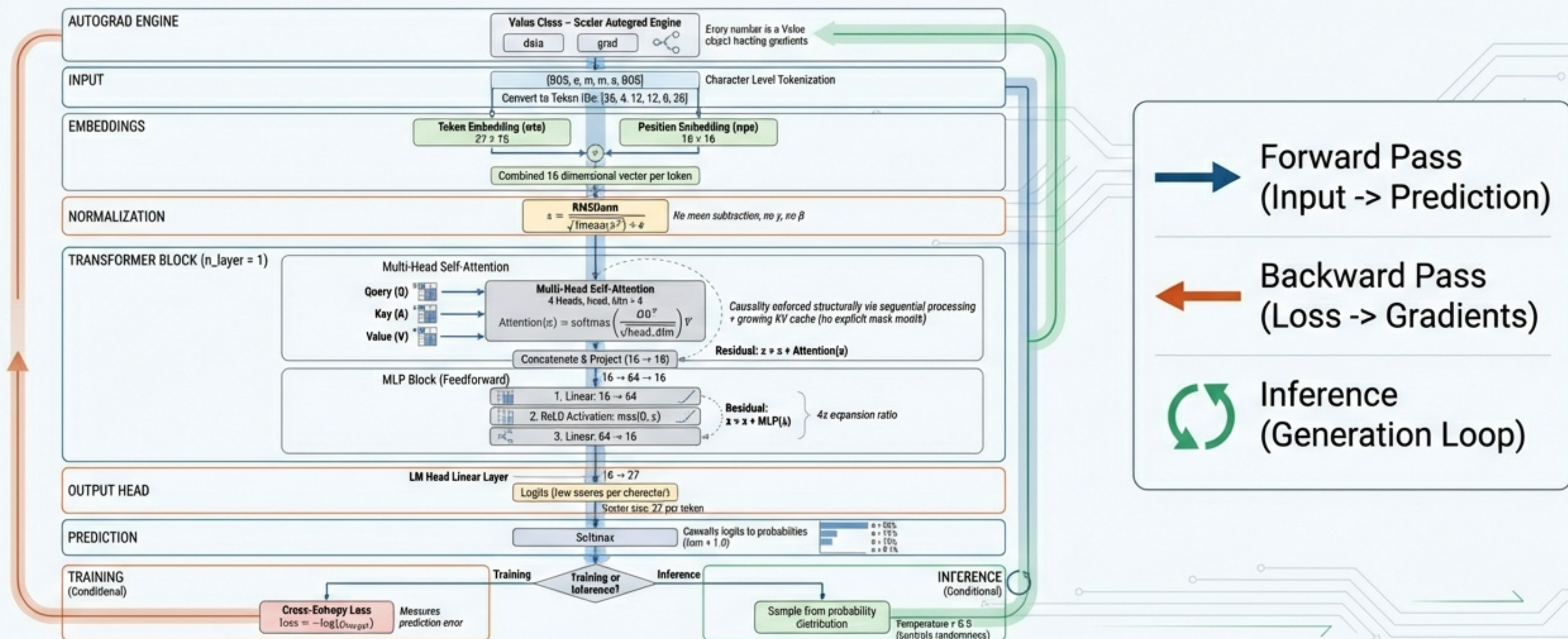


243

Total Lines of Code

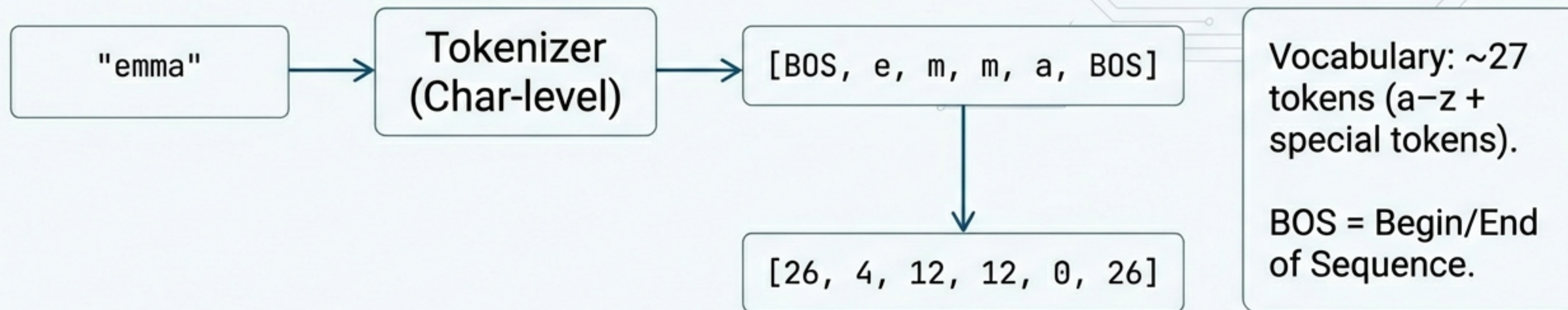
If you understand this, you understand the architecture of modern LLMs.

Big Picture Architecture



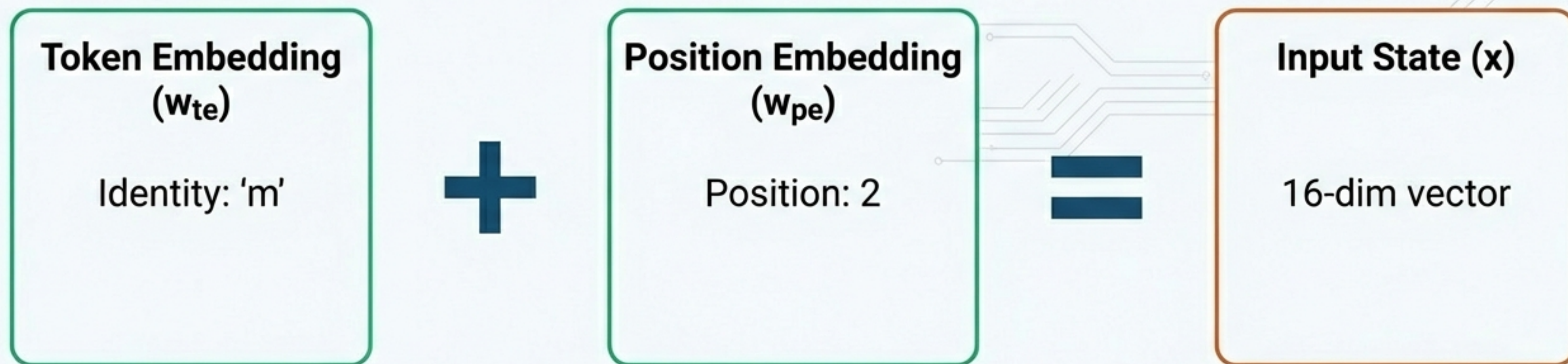
Step 1: Tokenization

From Text to Integers



Step 2: Embeddings

Identity + Position



$$x = wte[token_index] + wpe[position_index]$$

Step 3: RMSNorm

Stabilizing the Signal

$$x = \frac{x}{\sqrt{\text{var}(\text{mean}(x^2)) + \text{eps}}}$$

Key Differences vs LayerNorm

- No mean subtraction (centering)
- No learned bias parameters (beta)
- Pure scaling of variance
- Simpler and faster gradient calculation

Transformer Block Overview

Block (n_layer=1)

Multi-Head Attention
(Communication)



MLP Feedforward
(Computation)

Micro Configuration

- `n_embd = 16`
- `n_head = 4`
- `head_dim = 4`

Step 4: Multi-Head Self-Attention

Tokens talking to each other

Projections

- Query (Q)
- Key (K)
- Value (V)

The Math

$$\text{Attention}(\mathbf{x}) = \text{softmax}(\mathbf{QK}^T / \sqrt{\text{head_dim}}) * \mathbf{V}$$

Causality

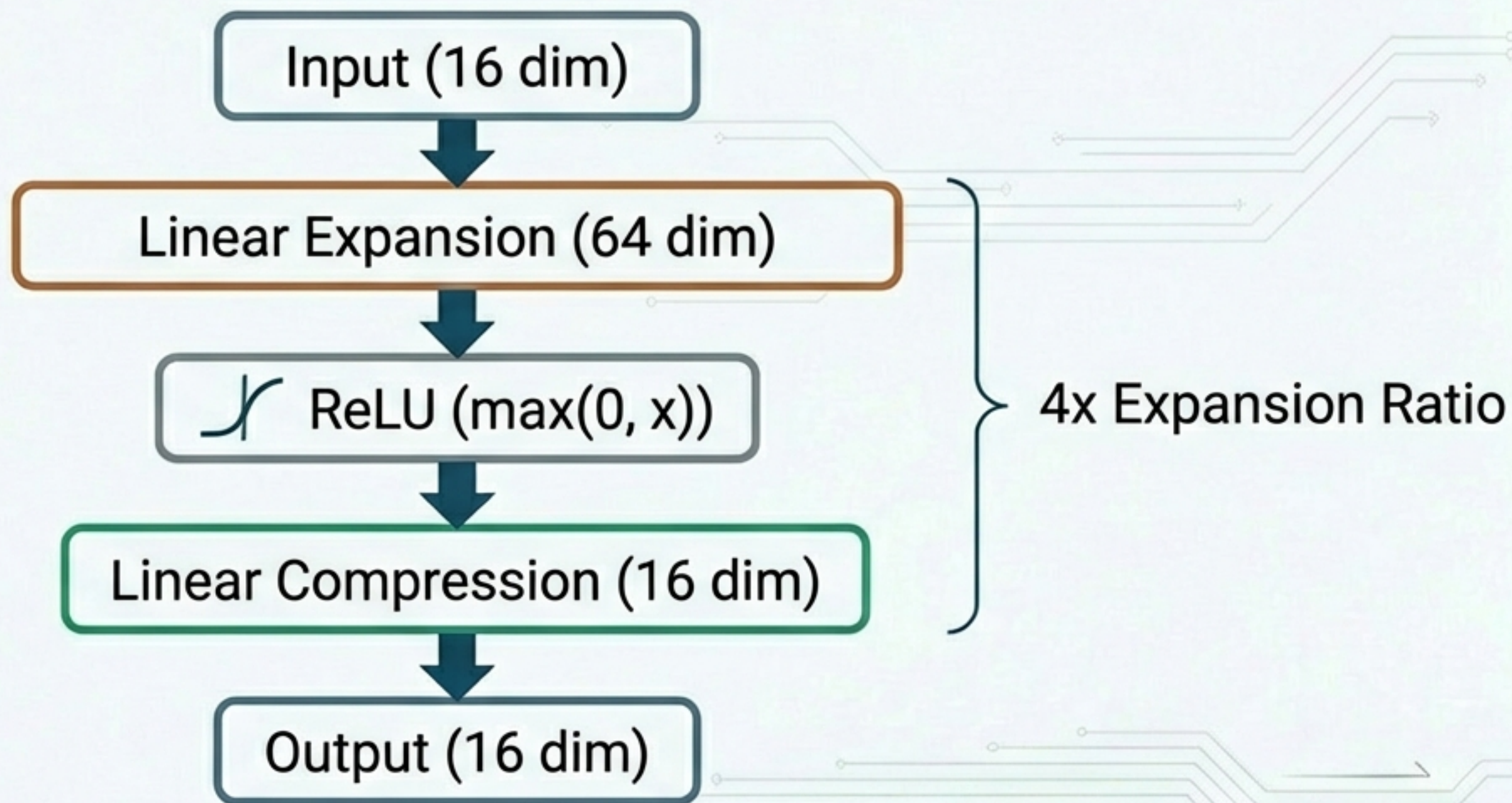
No explicit mask matrix. Causality is enforced structurally via sequential processing.

Residual Connection

$$\mathbf{x} = \mathbf{x} + \text{Attention}(\mathbf{x})$$

Step 5: MLP Block

Deep Thinking / Computation



$$\mathbf{x} = \mathbf{x} + \text{MLP}(\mathbf{x})$$

Step 6 & 7: LM Head + Softmax

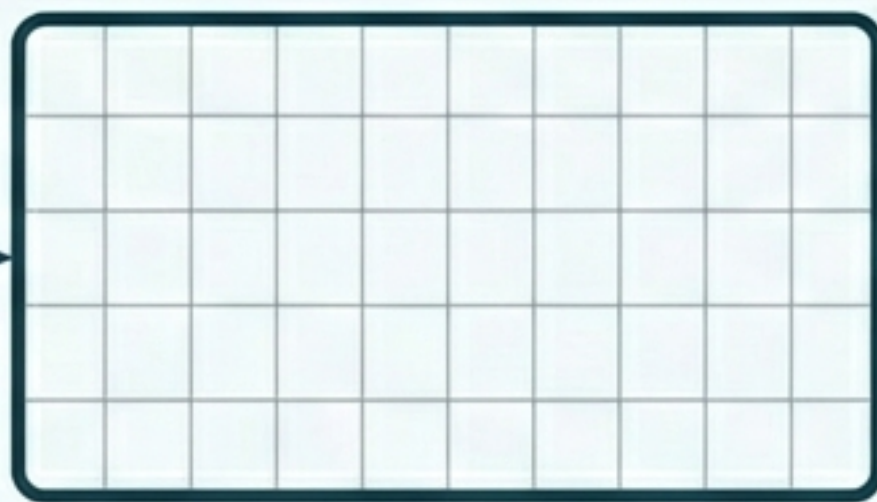
Projecting to Probabilities

Final State
(16-dim)



Linear Layer

Logits
(27-dim)



Softmax

a (60%)

o (20%)

z (0.1%)

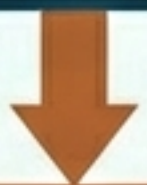
Sum of probabilities = 1.0

Final State
(16-dim)

Logits
(27-dim)

Step 8: The Loss Function

Measuring the Error


$$\text{loss} = -\log(\text{probability_of_target})$$

Model is Confident & Correct:

$p=1.0 \rightarrow \text{Loss}=0.0 \rightarrow \checkmark$

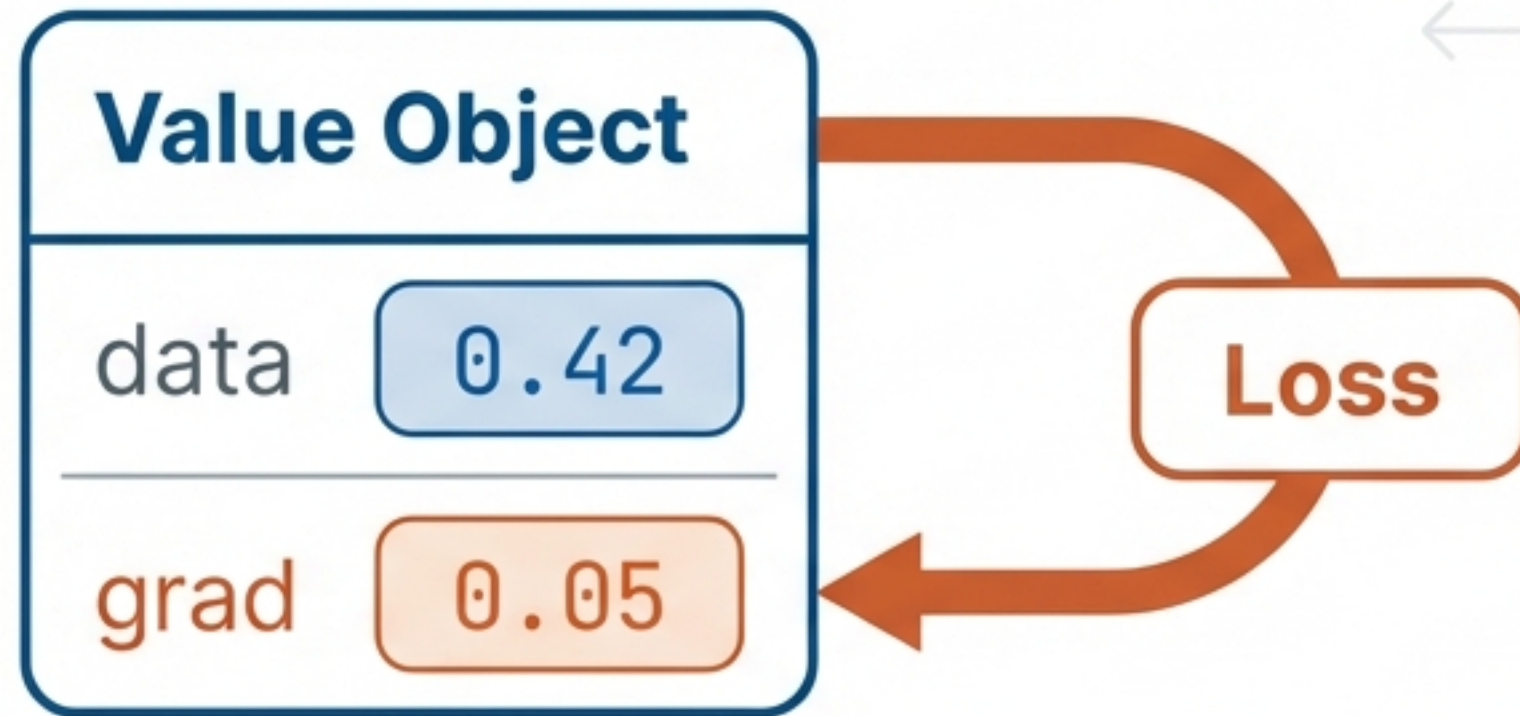
Model is Surprised:

$p=0.1 \rightarrow \text{Loss}=\text{High} \rightarrow !?$

We use Negative Log Likelihood to quantify how well the model predicted the actual next character.

Step 9: Backpropagation

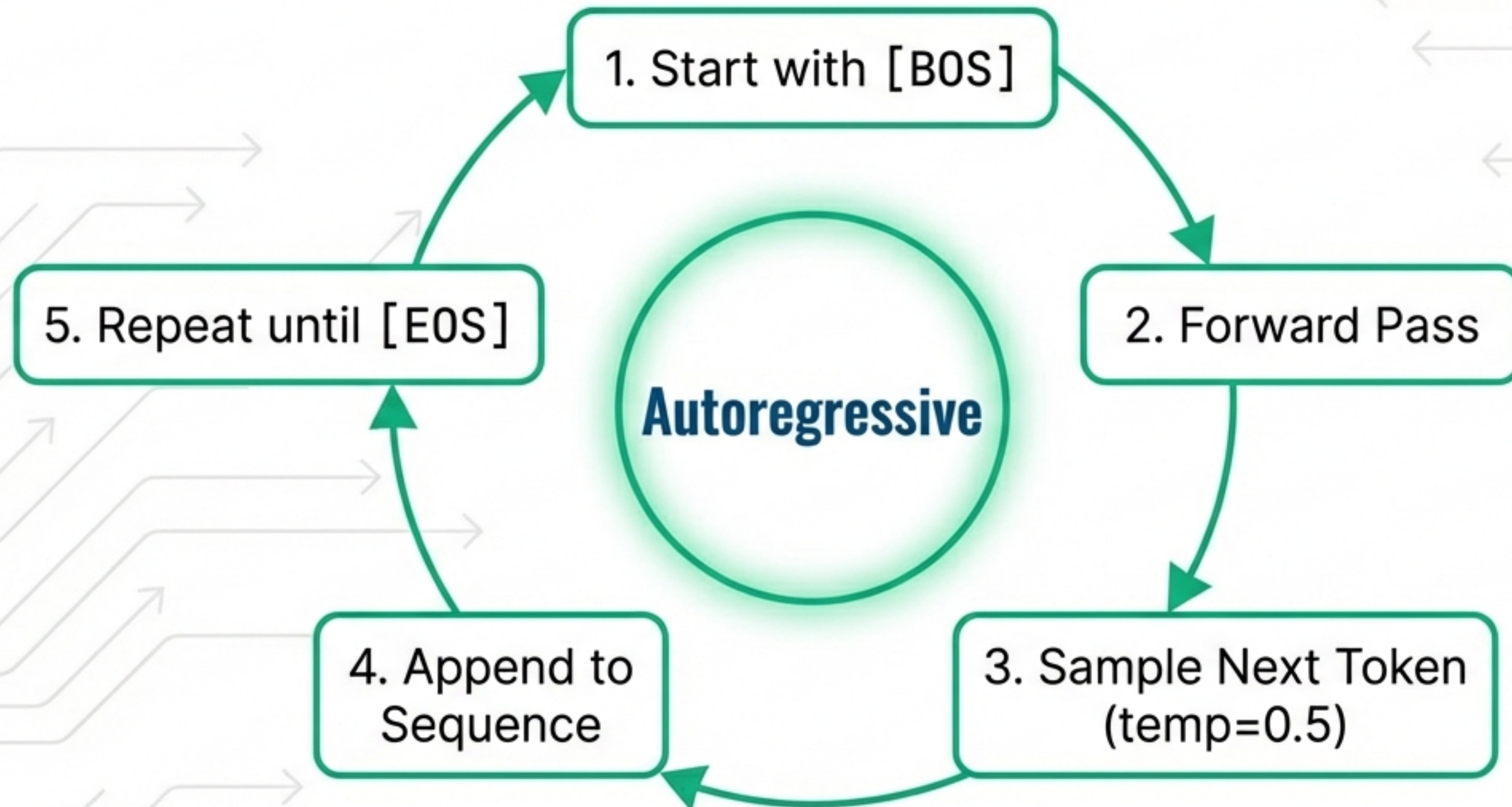
The Autograd Engine



- Topological sort of the computation graph
- Chain rule applied to every scalar
- Optimizer (Adam) updates parameters: $p.data += -lr * p.grad$

Inference: The Generation Loop

The Autoregressive Process of Text Generation



Parameter Breakdown

Total Parameters: 4,192

Scaling Factor		Impact on Parameters
Vocab Size	→	Linear Impact
Number of Layers	→	Linear Impact
Embedding Dimension	→	Quadratic Impact

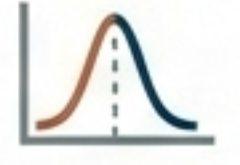
Most parameters live in the Embeddings and Linear projection layers.

What Makes microGPT Special

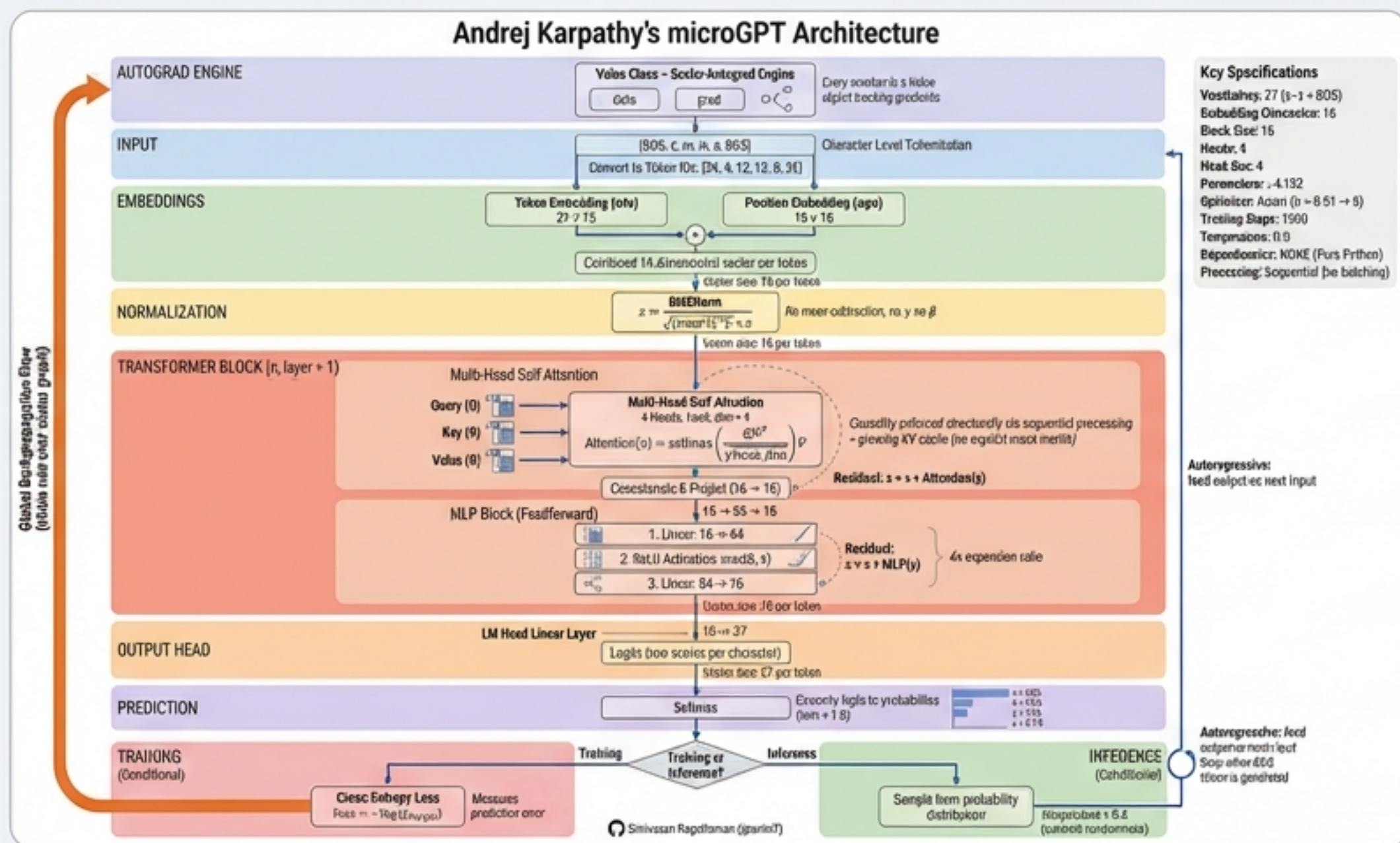
- ✓ Scalar Autograd Engine
- ✓ Pure Python (No GPU required)
- ✓ No Tensor Libraries
- ✓ No Learned Biases
- ✓ No LayerNorm parameters
- ✓ No explicit Mask Matrix

It proves the Transformer algorithm itself is remarkably small.

The Five Irreducible Ideas

- 1. Embed** Create numerical identity and position. 
- 2. Attend** Route information between tokens. 
- 3. Transform** Process information individually (MLP) 
- 4. Predict** Project to probabilities. 
- 5. Learn** Backpropagate error via chain rule. 

Conclusion



“The complexity of modern LLMs is not the algorithm — it’s the scale.”

microGPT Architecture